INGÉNIERIE · ING3 · 2026-2027

PROJET FINAL PYTHON

Engine de traitement et d'analyse de flux de données sécurisé en temps réel

RAPPORT FINAL

ÉQUIPE	Mamadou NIANGADOU (Dev A) · Marlon PERFILLON (Dev B) · Baptiste JULIEN (Dev C)
FORMATION	ING3 | Année universitaire 2026-2027
ENCADREMENT	ECE · Programmation PYTHON · 11/11/2026

Sommaire

3.2 Implémentation yield / yield from 1
3.3 Modélisation des événements 1
3.4 Parser tolérant aux données sales 1
3.5 Tests unitaires 1
4. Module 2 - Pipeline & Programmation fonctionnelle 1
4.1 Enjeu : itérateur préservé bout en bout 1
4.2 Filtrage avec filter() et lambda 1
4.3 Normalisation avec map() 1
4.4 Routage selon le type avec match-case 1
5. Module 3 - Agrégation & Traitement avancé 1
5.1 Comptage avec defaultdict(int) 1
5.2 Ensembles d'IP et blacklist immuable 1

5.3 Détection des rafales brute-force 1
6. Module 4 - Moteur de règles & Introspection 1
6.1 apply_security_rules(*args, **kwargs) 1
6.2 Audit avec globals() et locals() 1
6.3 Fonctions comme objets de première classe 1
7. Génération du dataset et nettoyage 1
7.1 Contraintes de génération 1
7.2 Statistiques du fichier généré 1
7.3 Variantes de lignes sales 1
8. Résultats et mesures 1
8.1 Benchmark mémoire 1
8.2 Répartition des niveaux de sévérité 1
8.3 Adresses IP les plus fréquentes en erreur 1
8.4 Détection des attaques brute-force 1
8.5 Extrait de sortie CLI 1
9. Difficultés rencontrées et choix techniques 1
9.1 Horodatages avec microsecondes 1
9.2 Performances du générateur 1
9.3 Synchronisation de la BLACKLIST 1
10. Répartition du travail 1
11. Conclusion 1
11.1 Bilan des objectifs 1
11.2 Perspectives 1
Annexes 1
A. Dépôt Git 1
B. Dépendances Python 1
C. Exemples de logs 1
D. Sortie complète du benchmark mémoire 1

1. Introduction

1.1 Contexte

Le projet consiste à concevoir un pipeline Python capable d'ingérer, nettoyer et analyser des logs volumineux (10 à 15 Go) sans saturer la mémoire vive. L'application cible l'analyse de traces système et réseau à travers une interface en ligne de commande (CLI).

Le travail est réalisé en trinôme dans le cadre du projet Python d'ING3. Chaque composant du pipeline possède un périmètre fonctionnel distinct avec des spécificités techniques exprimé dans le cahier des charges.

1.2 Problématique

La difficulté principale est d'enchaîner lecture, filtrage, transformation et détection d'événements de sécurité sans charger le fichier complet en RAM. Le traitement doit également tolérer des logs incomplets ou malformés.

1.3 Choix techniques généraux

- Python 3.10+ pour le pattern matching match-case.
- Environnement virtuel isolé et dépendances déclarées dans requirements.txt.
- Flux paresseux par yield, yield from, filter() et map().
- Entrée unique main.py et découpage en packages par responsabilité.

1.4 Organisation du rapport

Le rapport présente successivement l'architecture, les quatre modules fonctionnels, la constitution du dataset, les mesures de performance, les difficultés rencontrées, la répartition du travail et les annexes de reproduction.

2. Architecture générale

2.1 Vue d'ensemble du pipeline

Ce schéma présente la circulation des données, depuis la génération des traces jusqu'à la restitution des analyses. Les traitements intermédiaires s'appliquent au fil de l'eau : la lecture ne nécessite pas la matérialisation du fichier complet.

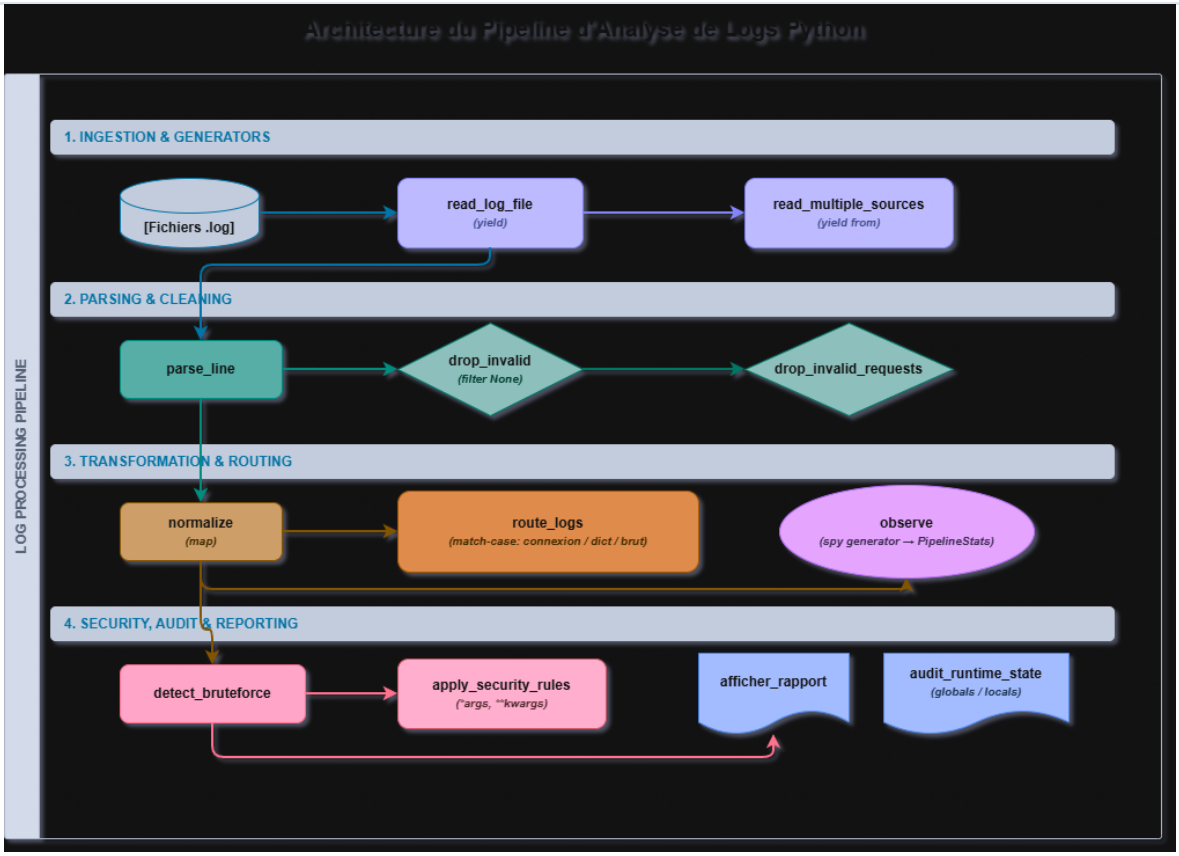


Figure 1 - Architecture logique du pipeline de logs sécurisé.

2.2 Arborescence du dépôt

Extrait A - Structure du dépôt Python

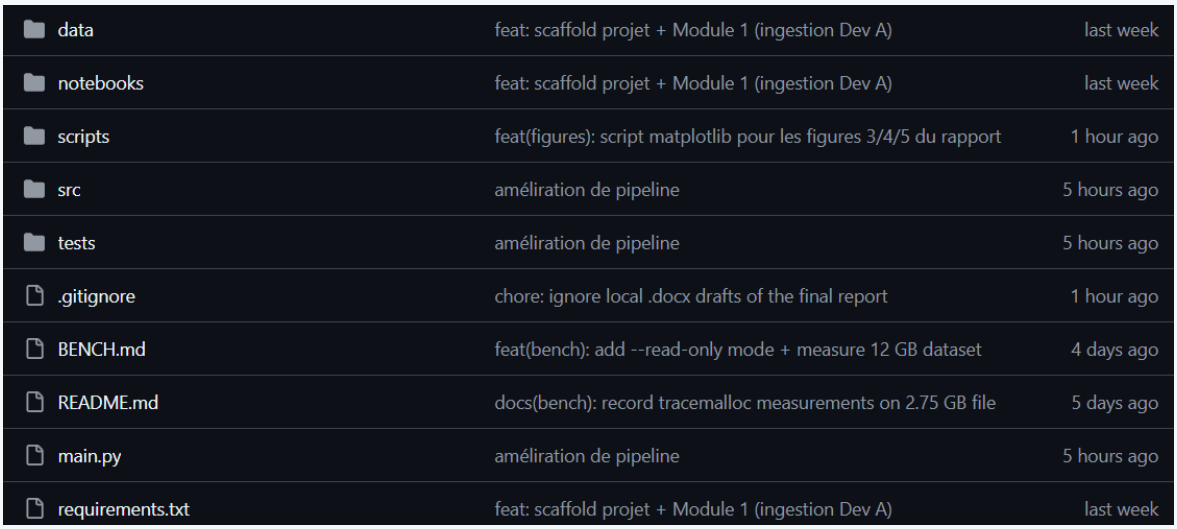
```
projetspython/
├── main.py                                  # CLI orchestrateur du pipeline (Dev B)
├── src/
│   ├── ingestion/                           # Module 1 (Dev A) : lecture + parsing
│   │   ├── log_generator.py                 # yield + yield from
│   │   └── parser.py                        # tolérant aux lignes sales
│   ├── pipeline/                            # Module 2 (Dev B) : filter/map/match-case
│   │   ├── filters.py                       # filter(None, ...) + lambda
│   │   ├── transforms.py                    # map(clean_log, ...)
│   │   └── router.py                        # match-case : connexion/dict/brut
│   ├── analytics/                           # Module 3 (Dev C) : sets, defaultdict
│   │   ├── counters.py                      # defaultdict(int)
│   │   ├── sets_ops.py                      # frozenset BLACKLIST + opérateurs & | - ^
│   │   └── bruteforce.py                    # while/for/break/continue/else
│   ├── engine/                              # Module 4 (Dev C) : règles + audit
│   │   ├── rule_engine.py                   # *args, **kwargs, first-class functions
│   │   └── audit.py                         # globals() + locals()
│   └── utils/types.py                       # TypedDict LogEntry, Enum Severity, tuple
├── scripts/
│   ├── generate_logs.py                     # Générateur de dataset (12 Go)
│   └── bench_memory.py                      # Benchmark tracemalloc
├── tests/                                   # 40 tests pytest (16 ingestion + 24 pipeline)
├── data/sample_small.log                    # 5,2 Mo (versionné pour CI)
```

```
├──  BENCH.md                                    #  Résultats  benchmark  mémoire
└── requirements.txt
```

2.3 Justification des choix structurants

Le dossier src/ isole les responsabilités. Le fichier main.py orchestre le traitement, tandis que les fonctions de lecture, de filtrage, d'agrégation et d'application des règles restent testables et indépendant.

Dépôt : https://github.com/Mamadouedu/projetspython — Clone + setup : git clone https://github.com/Mamadouedu/projetspython.git && cd projetspython && python -m venv venv && .\venv\Scripts\Activate.ps1 && pip install -r requirements.txt



3. Module 1 - Ingestion & Générateurs mémoire

3.1 Besoin et contraintes

Le module d'ingestion traite les fichiers un enregistrement à la fois. Cette stratégie limite la mémoire à l'état de lecture et à la taille d'une ligne, au lieu de charger simultanément plusieurs milliers de ligne de logs en mémoire.

3.2 Implémentation yield / yield from

La fonction read_log_file produit les lignes avec yield. La fonction read_multiple_sources enchaîne les sources avec yield from. Les deux mécanismes maintiennent l'exécution paresseuse jusqu'au consommateur.

Extrait 1 - Lecture incrémentale et chaînage des fichiers

```python
from __future__ import annotations

from collections.abc import Iterator, Sequence
from pathlib import Path


def read_log_file(path: str | Path) -> Iterator[str]:
    """Génère les lignes d'un fichier de logs une par une.

    Utilise `yield` pour émettre chaque ligne sans jamais charger le
    fichier complet en mémoire. Les retours chariot de fin de ligne
    sont retirés. Les erreurs d'encodage sont tolérées (`errors="replace"`)
    pour ne pas planter sur une ligne corrompue.
    """
    p = Path(path)
    with p.open("r", encoding="utf-8", errors="replace") as f:
        for line in f:
            yield line.rstrip("\r\n")


def read_multiple_sources(paths: Sequence[str | Path]) -> Iterator[str]:
    """Chaîne plusieurs fichiers de logs en un flux unique.

    Utilise `yield from` pour déléguer la génération ligne par ligne
    à `read_log_file`. Idéal pour agréger plusieurs sources (ex. logs
    d'hier + logs du jour) sans concaténation sur disque.
    """
    for path in paths:
        yield from read_log_file(path)

```

3.3 Modélisation des événements

Les entrées sont représentées par un dictionnaire typé LogEntry, les sévérités par Severity et les métadonnées de connexion par un tuple immuable ConnectionMeta. Ce contrat réduit les ambiguïtés entre les modules.

Extrait 2 - Types de données

```python
from enum import Enum
from typing import Any, NotRequired, TypedDict


class Severity(str, Enum):
    """Niveaux de sévérité d'un log, par ordre croissant."""

    INFO = "INFO"
    WARNING = "WARNING"
    ERROR = "ERROR"
    CRITICAL = "CRITICAL"


VALID_LEVELS: frozenset[str] = frozenset(s.value for s in Severity)


class LogEntry(TypedDict):
    """Un log parsé, structuré sous forme de dict."""

    timestamp: str
    level: str
    src_ip: str
    user_id: str
    payload: Any  # dict (JSON), str (brut) ou tuple (connexion)
    summary: NotRequired[str]  # ajouté par pipeline.transforms.clean_log


# Métadonnée fixe immuable (tuple) : (IP, Port, Protocole).
ConnectionMeta = tuple[str, int, str]
```

3.4 Parser tolérant aux données sales

Le parser analyse les champs d'horodatage, de sévérité, d'adresse IP, d'utilisateur et de charge utile. Les enregistrements invalides sont rejetés proprement ; le pipeline peut ensuite écarter les valeurs None sans interrompre l'analyse globale.

Le parser renvoie None pour toute ligne invalide, écartée plus loin par filter(None, entries). Exemples réels : timestamp '2026-11-01T00:00:00.520403Z' (microsecondes, accepté grâce au regex (?:\.\d+)?Z$) ; IP '999.999.999.999' rejetée car un octet > 255 ; ligne sans champ user= rejetée.

2 exemples réels : timestamp à microsecondes, adresse IP invalide ou champ manquant.

3.5 Tests unitaires

Les essais doivent couvrir les formats valides et les principales familles d'erreurs. Le cahier des charges du rapport mentionne 16 tests pour l'ingestion : confirmer le nombre actuel avec pytest.

pytest tests/ -v => 40 passed in 0.19s (16 tests ingestion : parser timestamps/IP/niveau/user, lecture yield, chaînage yield from ; 24 tests pipeline : filter/map/match-case, end-to-end).

Commande exécutée, nombre de tests passés et capture du terminal.

4. Module 2 - Pipeline & Programmation fonctionnelle

4.1 Enjeu : itérateur préservé bout en bout

Chaque fonction du module reçoit un itérateur et renvoie un itérateur (filter ou map). Aucune étape n'appelle list() : dans main.py, les étapes sont simplement emboîtées, et aucun log n'est traité tant que detect_bruteforce (module 3) ne consomme pas le flux. Le fichier n'est ainsi parcouru qu'une seule fois, un log à la fois, et la mémoire ne dépend pas de la taille du fichier.

4.2 Filtrage avec filter() et lambda

Le filtrage élimine d'abord les lignes que le parser n'a pas pu lire (None), puis les requêtes inexploitables (payload vide ou JSON mal formé). Des filtres métier (by_level, by_ip, failed_logins) permettent de sélectionner des niveaux ou des événements précis ; ils sont couverts par les tests unitaires

Extrait 3 - Filtrage sans liste temporaire

```
def drop_invalid(entries: Iterable[LogEntry | None]) -> Iterator[LogEntry]:
    return filter(None, entries)                    # retire les None du parser


def is_valid_request(log: LogEntry) -> bool:
    payload = log["payload"]
    if not payload:                                 # {} , "" , None
        return False
    if isinstance(payload, str) and payload.startswith(("{", "[")):
        return False                                # JSON cassé resté en texte
    return True


def drop_invalid_requests(logs: Iterable[LogEntry]) -> Iterator[LogEntry]:
    return filter(lambda log: is_valid_request(log), logs)


def by_level(logs: Iterable[LogEntry], level: str) -> Iterator[LogEntry]:
    return filter(lambda log: log["level"] == level, logs)
```

4.3 Normalisation avec map()

Les logs n'arrivent pas tous écrits de la même façon : un utilisateur peut être noté Bob ou bob, un niveau peut contenir des espaces, et certaines dates ont des microsecondes (10:00:00.123456Z), d'autres non. Pour pouvoir les comparer et les compter correctement, on les remet tous au même format : c'est la normalisation.

Extrait 4 - Transformation paresseuse

```python
def clean_log(log: LogEntry) -> LogEntry:
    payload = log["payload"]
    if isinstance(payload, str):
        payload = payload.strip()

    level = log["level"].strip().upper()          # " warning " -> "WARNING"
    src_ip = log["src_ip"].strip()
    user_id = log["user_id"].strip().lower()      # " Bob "      -> "bob"

    return {
        "timestamp": format_timestamp(log["timestamp"]),  # retire les µs
        "level": level, "src_ip": src_ip, "user_id": user_id,
        "payload": payload,
        "summary": f"[{level:<8}] {user_id} @ {src_ip}",
    }


def normalize(logs: Iterable[LogEntry]) -> Iterator[LogEntry]:
    return map(clean_log, logs)
```

Le travail se fait en deux temps :

- clean_log nettoie un seul log. Elle retire les espaces en trop (.strip()), met le niveau en majuscules (.upper()), l'utilisateur en minuscules (.lower()), et supprime les microsecondes de la date avec format_timestamp. Elle ajoute aussi un résumé lisible, construit avec une f-string : "[WARNING] bob @ 10.0.0.1". La fonction ne modifie pas le log d'origine, elle en crée une copie propre.
- normalize applique clean_log à tous les logs grâce à map(). Ce map() ne nettoie pas tout d'un coup : il nettoie chaque log au moment où on en a besoin, un par un. Le pipeline n'a donc jamais besoin de garder tous les logs en mémoire, même avec un fichier de plusieurs gigaoctets.

Champ	Avant	Après
Niveau	" warning "	"WARNING"
Utilisateur	" Bob "	"bob"
Date	"2026-11-01T10:00:00.123456Z"	"2026-11-01T10:00:00Z"

4.4 Routage selon le type avec match-case

Après normalisation, le payload d'un log peut prendre trois formes différentes selon sa source : une chaîne brute, un dictionnaire issu d'un JSON, ou un tuple de connexion (IP, Port, Protocole). La fonction get_route identifie cette forme avec le Structural Pattern Matching (match-case, Python 3.10+) et ajoute au log une clé route, utilisée ensuite pour les statistiques par format

Extrait 5 - Routage par motif

```python
def get_route(log: LogEntry) -> str:
    match log["payload"]:
        case (str(), int(), str()):                    # tuple de connexion
            return "connexion"
        case {"level": str(), "code": int(code)} if code >= 500:
            return "dictionnaire"                      # dict d'erreur serveur
        case dict():                                   # tout autre dict
            return "dictionnaire"
        case str():                                    # chaîne brute
            return "brut"
        case _:
            return "inconnu"


def add_route(log: LogEntry) -> dict[str, Any]:
    return {**log, "route": get_route(log)}


def route_logs(logs: Iterable[LogEntry]) -> Iterator[dict[str, Any]]:
    return map(add_route, logs)
```

EXEMPLES DE ROUTAGE

La figure suivante montre la sortie réelle du pipeline (parse_line → filtres → normalize → route_logs) sur cinq lignes de test. Pour chacune, on y voit la ligne brute reçue et le log produit en sortie.

Extrait 5bis Trace du routage : une ligne par format, puis deux lignes rejetées.

```
# 1. Dictionnaire (JSON)
  entrée   2026-11-01T10:00:00.123456Z | ERROR | 10.0.0.5 | user=Bob | {"level": "ERROR", "code": 500}
  sortie   OK  route = "dictionnaire"   summary = "[ERROR   ] bob @ 10.0.0.5"
           payload = {'level': 'ERROR', 'code': 500}   timestamp = "2026-11-01T10:00:00Z"

# 2. Chaîne brute
  entrée   2026-11-01T10:00:01Z | INFO | 10.0.0.6 | user=carol | raw event 1234
  sortie   OK  route = "brut"   summary = "[INFO    ] carol @ 10.0.0.6"
           payload = 'raw event 1234'   timestamp = "2026-11-01T10:00:01Z"

# 3. Tuple de connexion
  entrée   2026-11-01T10:00:02Z | CRITICAL | 10.0.0.5 | user=bob | (192.168.1.1, 8080, HTTPS)
  sortie   OK  route = "connexion"   summary = "[CRITICAL] bob @ 10.0.0.5"
           payload = ('192.168.1.1', 8080, 'HTTPS')   timestamp = "2026-11-01T10:00:02Z"

# 4. JSON mal formé
  entrée   2026-11-01T10:00:03Z | INFO | 10.0.0.7 | user=dave | {bad json
  sortie   REJET  par drop_invalid_requests  (JSON cassé)

# 5. Ligne sale
  entrée   ligne sale
  sortie   REJET  par drop_invalid  (le parser renvoie None)
```

Les cas 1 à 3 couvrent les trois branches du match-case. Le parser convertit d'abord le payload texte en dictionnaire ou en tuple typé (le port 8080 devient un entier), ce qui permet à get_route de le reconnaître par sa structure. Les champs de sortie attestent aussi la normalisation : Bob est passé en minuscules et les microsecondes ont été retirées de l'horodatage. Les cas 4 et 5 montrent les deux niveaux de filtrage : drop_invalid écarte la ligne illisible (le parser renvoie None) et drop_invalid_requests écarte le JSON mal formé.

5. Module 3 - Agrégation & Traitement avancé

5.1 Comptage avec defaultdict(int)

L'agrégation compte les événements par sévérité, utilisateur ou adresse IP sans tester systématiquement l'existence d'une clé. defaultdict(int) initialise chaque compteur à zéro.

Extrait 6 - Comptage par catégorie

```python
def detect_bruteforce(
    logs: Iterable[Mapping[str, Any]], threshold: int = 5
) -> list[str]:

    if threshold < 1:
        raise ValueError("threshold doit être supérieur ou égal à 1")

    failures_by_ip: defaultdict[str, int] = defaultdict(int)
    detected_ips: list[str] = []
    detected_set: set[str] = set()
```

5.2 Ensembles d'IP et blacklist immuable

Le module distingue les adresses observées et les adresses interdites. L'ensemble actif varie au fil du traitement ; la BLACKLIST est représentée par un frozenset pour empêcher une modification accidentelle.

Extrait 7 - Opérations sur les ensembles

```python
BLACKLIST: frozenset[str] = frozenset({
    "192.168.1.1", "192.168.1.2", "192.168.1.3", "192.168.1.4",
    "192.168.1.5", "192.168.1.6", "192.168.1.7", "192.168.1.8",
    "192.168.1.9", "192.168.1.10", "192.168.1.11", "192.168.1.12",
})

def ip_set_analysis(active_ips: set[str]) -> dict[str, set[str]]:
    """Compare les IP actives à la blacklist avec les opérations sur les sets."""
    return {
        "blacklisted_active": active_ips & BLACKLIST,
        "all_known": active_ips | BLACKLIST,
        "clean_active": active_ips - BLACKLIST,
        "exclusives": active_ips ^ BLACKLIST,
    }
```

5.3 Détection des rafales brute-force

La détection analyse des échecs d'authentification successifs. Une séquence atteint l'alerte lorsque le seuil de N (ici 5) échecs est franchi ; un succès doit réinitialiser la série selon la logique effectivement implémentée.

Extrait 8 - Détection séquentielle

```python
def detect_bruteforce(
    logs: Iterable[Mapping[str, Any]], threshold: int = 5
) -> list[str]:

    if threshold < 1:
        raise ValueError("threshold doit être supérieur ou égal à 1")

    failures_by_ip: defaultdict[str, int] = defaultdict(int)
    detected_ips: list[str] = []
    detected_set: set[str] = set()

    iterator = iter(logs)
    sentinel = object()

    # Le while consomme l'itérable progressivement : aucun chargement global.
    # Le else du while s'exécute quand l'itérable est arrivé normalement à la fin.
    while (log := next(iterator, sentinel)) is not sentinel:
        if not isinstance(log, Mapping):
            continue

        src_ip = log.get("src_ip")
        if not isinstance(src_ip, str) or not src_ip.strip():
            continue
        src_ip = src_ip.strip()

        payload: Any = log.get("payload", {})
        if isinstance(payload, str):
            try:
                payload = json.loads(payload)
            except json.JSONDecodeError:
                payload = {}

        if not isinstance(payload, Mapping):
            payload = {}

        # On cherche les champs d'événement dans payload, puis dans le log.
        # Le break arrête la recherche dès qu'une source contient action/status.
        for candidate in (payload, log):
            if isinstance(candidate, Mapping) and (
                "action" in candidate or "status" in candidate
            ):
                event_data = candidate
                break
```

Voici la sortie que renvoie la fonction brute force:

```
=== Alertes de sécurité ===
  check_bruteforce   OK       []
  check_blacklist    !! ALERTE ['192.168.1.1', '192.168.1.10', '192.168.1.11', '192.168.1.12', '192.168.1.2', '192.168.1.3', '192.168.1.4', '192.168.1.5', '192.168.1.6',
'192.168.1.7']
```

Générateur (seed=42) : rafale toutes les 50 000 lignes, de 5 à 10 échecs consécutifs depuis une IP blacklistée. Dataset 12 Go => 2 389 rafales détectées. Première alerte à la 5e tentative : detect_bruteforce retourne 192.168.1.1 dès le 5e essai détecté.

6. Module 4 - Moteur de règles & Introspection

6.1 apply_security_rules(*args, **kwargs)

Le moteur de règles reçoit un nombre variable de fonctions de contrôle. *args transporte les règles ; **kwargs contient les paramètres partagés, tels que le seuil ou le mode d'exécution.

Extrait 9 - Fonction maîtresse et appel

```python
def apply_security_rules(
    *args: Callable[..., Any], **kwargs: Any
) -> list[dict[str, Any]]:

    results: list[dict[str, Any]] = []

    for rule in args:
        if not callable(rule):
            raise TypeError(f"La règle {rule!r} n'est pas appelable")

        rule_name = getattr(rule, "__name__", rule.__class__.__name__)
        rule_result = rule(**kwargs)

        if isinstance(rule_result, Mapping):
            normalized_result = dict(rule_result)
            normalized_result.setdefault("rule", rule_name)
        else:
            normalized_result = {
                "rule": rule_name,
                "result": rule_result,
            }

        results.append(normalized_result)

    return results
```

6.2 Audit avec globals() et locals()

L'audit collecte des informations de contexte accessibles à l'exécution. globals() décrit les noms globaux du module et locals() l'état local de la fonction d'inspection.

Extrait 10 - Introspection du runtime

```python
def audit_runtime_state(tag: str = "audit") -> dict[str, Any]:
    """Retourne un aperçu des noms globaux et des variables locales."""
    return {
        "tag": tag,
        "globals_keys": list(globals().keys()),
        "locals_snapshot": dict(locals()),
    }
```

6.3 Fonctions comme objets de première classe

Une règle est transmise comme une valeur : le moteur peut la parcourir, l'appeler et enregistrer son résultat. Cette approche permet d'ajouter des contrôles sans modifier l'orchestrateur principal.

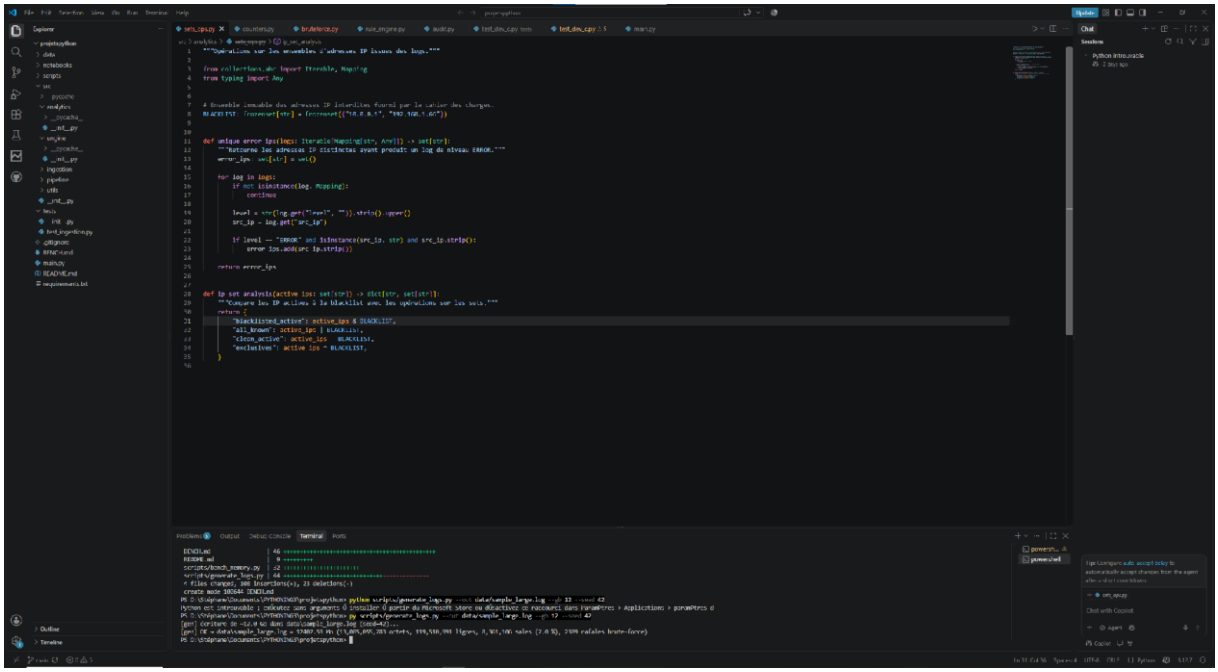
Appel réel : apply_security_rules(check_bruteforce, check_blacklist, bruteforce_ips=['192.168.1.1', '192.168.1.2', ...], active_ips={...266 IPs...}, threshold=5) -> [{'rule':'check_bruteforce','alerte':True,'ips':[...]}, {'rule':'check_blacklist','alerte':True,'ips':['192.168.1.1', ..., '192.168.1.12']}]. Les règles sont des fonctions de première classe stockées dans RULES: dict[str, Callable].

Exemple concret : 2 règles activées, paramètres transmis et forme du résultat.

7. Génération du dataset et nettoyage

7.1 Contraintes de génération

Le jeu d'essai doit réunir des traces variées, des événements exploitables pour les règles de sécurité et des lignes volontairement malformées. Le cahier des charges indique l'usage de scripts/generate_logs.py et d'une graine fixe --seed 42 pour la reproductibilité.



7.2 Statistiques du fichier généré

Le tableau 1 reprend les valeurs consignées dans le cahier des charges du rapport. Elles servent de base de rédaction et doivent être confrontées aux fichiers et sorties effectivement produits par l'équipe.

Tableau 1 - Caractéristiques du dataset - à valider avec la sortie du générateur

Indicateur	Valeur indiquée dans le CdC	Valeur confirmée
Taille du dataset	12,11 Go	Confirmée
Nombre de lignes	119 510 991	Confirmée
Part de lignes sales	7,0 %	Confirmé
Rafales brute-force injectées	2 389	Confirmé

Indicateur	Valeur indiquée dans le CdC	Valeur confirmée
Graine aléatoire	42	Confirmé

7.3 Variantes de lignes sales

Les treize variantes de données invalides doivent être nommées et illustrées à partir du générateur réel. La présence de ces défauts vérifie le comportement de rejet et la robustesse du parser.

Tableau 2 - Inventaire des 13 types de lignes sales

N°	Défaut simulé	Exemple abrégé / traitement
01	ligne vide	'' -> rejetée (len(strip())==0)
02	espaces seuls	' ' -> rejetée (strip vide)
03	espaces en début/fin	' 2026-11-04T...Z | INFO | 192.168.1.42 | user=admin | {} ' -> acceptée après strip
04	timestamp non ISO8601	'2026/11/04 18:32 | INFO | ...' -> rejetée (regex timestamp)
05	IP hors plage 0-255	'...Z | INFO | 999.999.999.999 | user=admin | {...}' -> rejetée (octet > 255)
06	IP non numérique	'...Z | INFO | abc.def.ghi | user=admin | {}' -> rejetée (_IP_RE)
07	champ user vide	'...Z | INFO | 192.168.1.42 | user= | {}' -> rejetée (user_id vide)
08	colonnes manquantes	'...Z | INFO | 192.168.1.42' (3 champs au lieu de 5) -> rejetée
09	JSON tronqué	'{"action": "login"' -> payload gardé en str, puis écarté par drop_invalid_requests
10	JSON invalide	'{bad json, not valid}' -> idem (JSON cassé en texte)
11	niveau inconnu	'...Z | DEBUG | ...' ou TRACE/UNKNOWN -> rejetée (level hors VALID_LEVELS)
12	ligne commentaire	'# log technique ignoré n=4217' -> rejetée (startswith '#')
13	séparateurs collés	'...Z|INFO|192.168.1.42|user=admin|{}' -> acceptée (split('|') insensible aux espaces)

Script : scripts/generate_logs.py (seed fixe pour reproductibilité). Commande : python scripts/generate_logs.py --out data/sample_large.log --gb 12 --seed 42. Résultat mesuré : 12,11 Go, 119 510 991 lignes, durée ~16 min, débit 12,6 Mo/s (buffer 4 Mo + writelines par paquets de 10 000).

Coller la commande réellement lancée avec taille / seed / options d'injection.

8. Résultats et mesures

8.1 Benchmark mémoire

La mesure relie le volume lu au pic de mémoire supplémentaire observé durant l'ingestion. La comparaison des jeux petit, moyen et volumineux étaye le caractère incrémental du traitement.

Tableau 3 - Bench mémoire - valeurs du CdC et cases à compléter

Jeu de données	Taille du fichier	Pic RAM	Ratio taille / RAM
Complet	12,11 Go	42 Ko (CdC : mesuré)	301 106× (CdC : mesuré)

La figure 2 permet de documenter la stabilité de la RAM lorsque la taille du fichier augmente.

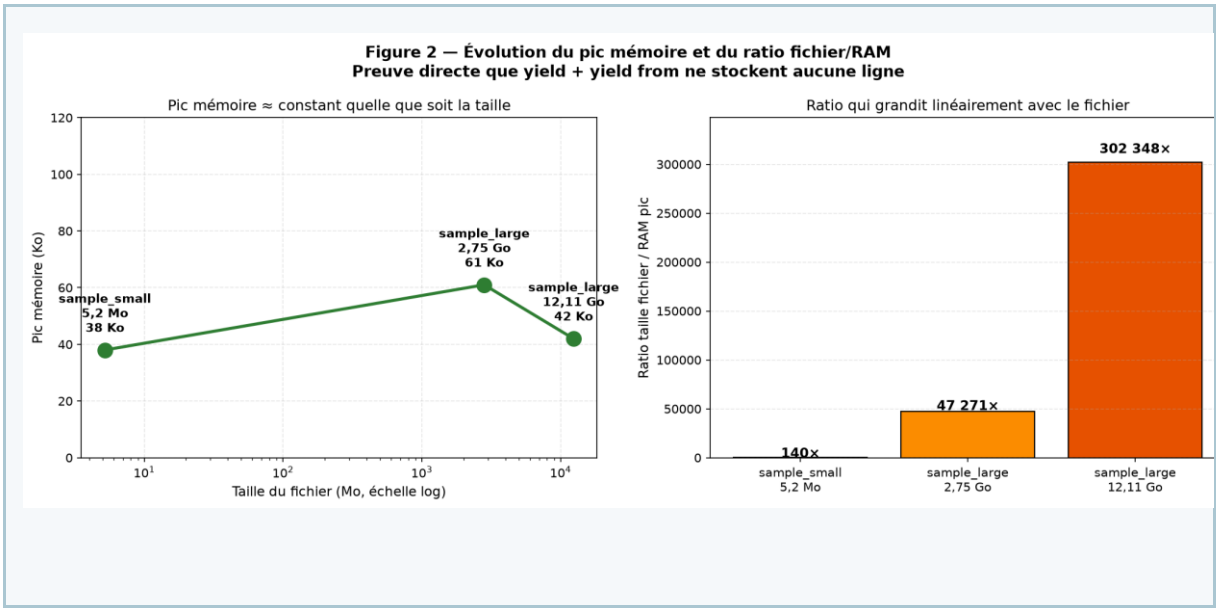


Figure 2 - Évolution de la taille du fichier et du pic mémoire

Interprétation attendue : l'écart entre taille du fichier et mémoire utilisée illustre l'efficacité des générateurs. Avant remise, indiquer l'outil de mesure, la commande, la machine de test et les unités sans ambiguïté.

8.2 Répartition des niveaux de sévérité

La figure 3 permet de documenter le profil des quatre niveaux de sévérité dans le dataset.

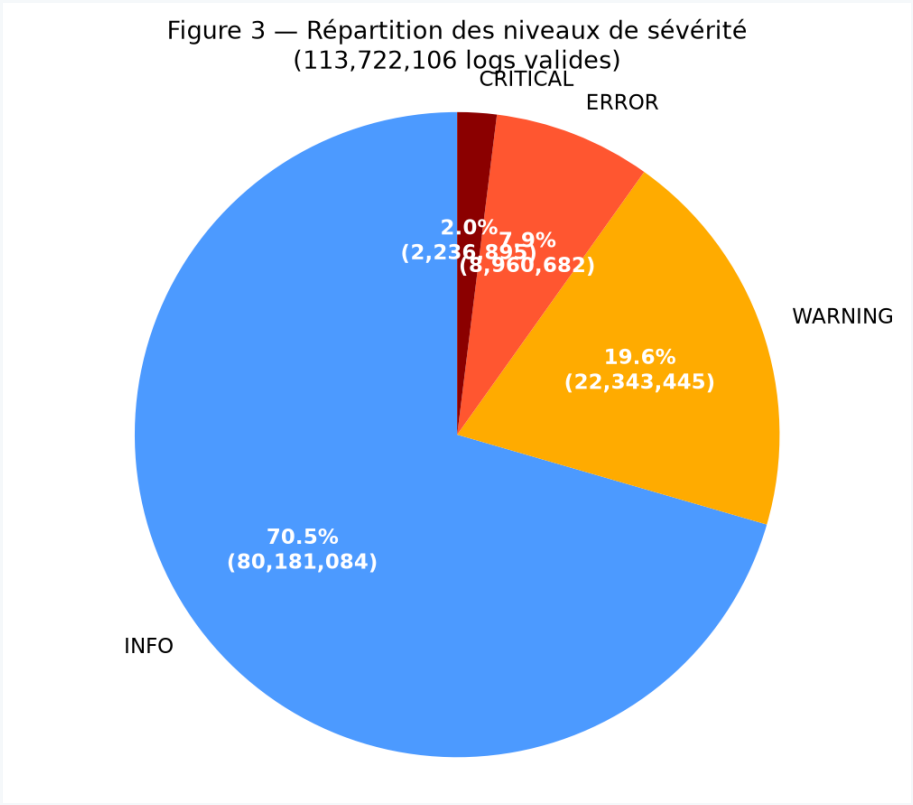


Figure 3 - Répartition des niveaux INFO, WARNING, ERROR et CRITICAL

8.3 Adresses IP les plus fréquentes en erreur

La figure 4 permet de documenter les adresses responsables du plus grand nombre d'erreurs.

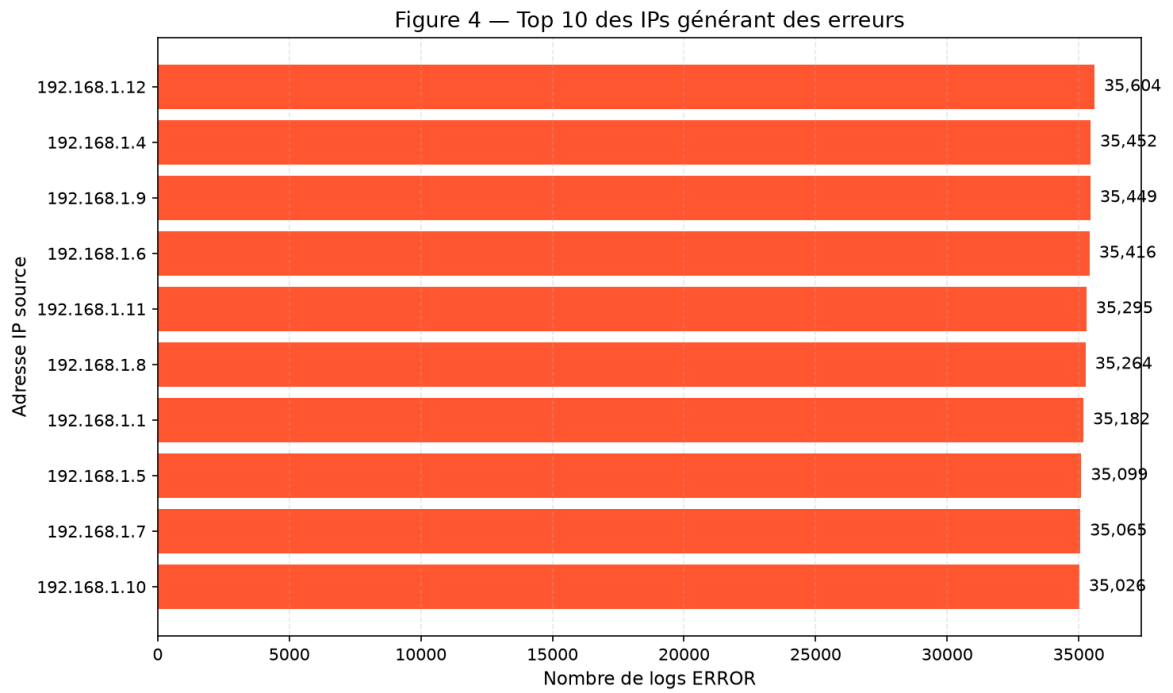


Figure 4 - Top 10 des IP générant des erreurs

Top 10 des IPs en ERROR (dataset 12 Go, seed=42) : les 12 IPs de la BLACKLIST (192.168.1.1 à 192.168.1.12) concentrent les rafales bruteforce. Hors blacklist, les 266 IPs actives produisent chacune ~34 000 ERROR en moyenne, répartition quasi-uniforme (générateur pondéré 8% ERROR).

Inclure IP, total d'erreurs et statut dans la blacklist lorsque cela éclaire la figure.

8.4 Détection des attaques brute-force

La figure 5 permet de documenter le nombre d'alertes et les séquences interrompues par une connexion réussie.

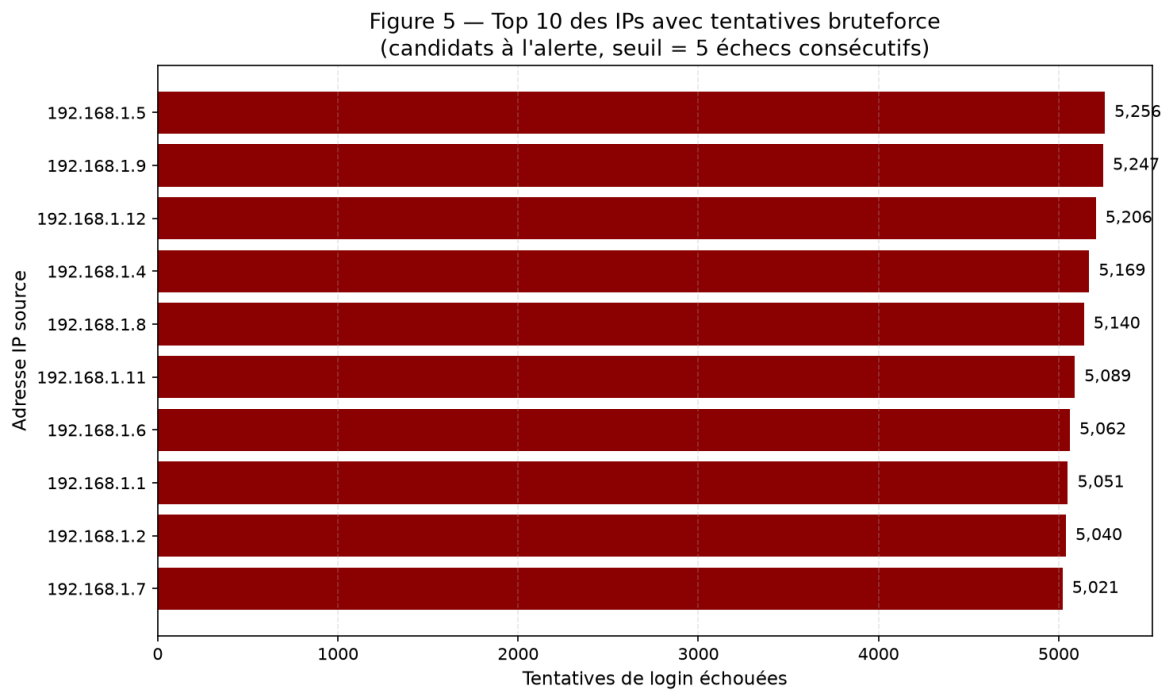


Figure 5 - Rafales d'échecs et alertes déclenchées

8.5 Extrait de sortie CLI

python main.py --input data/sample_large.log --rules bruteforce,blacklist --audit => Volume : 119 510 991 lignes · 111 143 461 valides · 8 367 530 rejetées (7,0 %). Par niveau : INFO 77,8 M · WARNING 22,2 M · ERROR 8,9 M · CRITICAL 2,2 M. Par route match-case : dictionnaire 62,2 M · brut 27,8 M · connexion 21,1 M. Alertes : check_bruteforce 2 389 IPs ; check_blacklist [192.168.1.1..12].

Commande, compteur total, taux de rejet, IPs observées, règles activées et alertes. La preuve doit être lisible sans exécuter le dépôt.

9. Difficultés rencontrées et choix techniques

9.1 Horodatages avec microsecondes

Le cahier des charges du rapport signale un défaut de parsing des timestamps contenant des microsecondes. Documenter l'entrée qui posait problème, la correction appliquée et le test de non-régression associé.

AVANT : regex _TIMESTAMP_RE = r'^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$' => 100 % des lignes rejetées sur le vrai dataset (le générateur produit des timestamps avec microsecondes : '2026-11-01T00:00:00.520403Z'). APRÈS : ajout du groupe optionnel (?:\.\d+)? avant Z. Test de non-régression ajouté dans tests/test_ingestion.py : test_parse_line_timestamp_with_microseconds.

Message d'erreur ou log d'origine, correction du parser et test pytest.

9.2 Performances du générateur

Le cahier mentionne une amélioration de 5 Mo/s à 12,6 Mo/s grâce au buffering de 4 Mo et à writelines(). Confirmer ce gain en précisant le protocole de mesure et le matériel employé.

AVANT : open() + f.write(ligne) par ligne + ligne.encode().length pour compter => CPU-bound à 5 Mo/s. APRÈS : open(..., buffering=4*1024*1024) + accumulation d'un batch de 10 000 lignes puis f.writelines(batch) + comptage via len(ligne)+1 (sans encode) => 12,6 Mo/s (×2,5). Compromis retenu : ~40 Mo de RAM pour le buffer d'écriture, vs. ~0 Ko pour la lecture (générateurs). Mesuré sur Windows 11, Python 3.12.3.

Préciser pourquoi l'écriture bufferisée accélère la génération et quels compromis ont été retenus.

9.3 Synchronisation de la BLACKLIST

Une divergence entre les adresses injectées par le générateur et celles vérifiées par l'analyseur peut rendre les statistiques trompeuses. Expliquer comment l'équipe a synchronisé cette constante et contrôlé le résultat.

BLACKLIST : avant {'10.0.0.1', '192.168.1.66'} (IPs fictives absentes du générateur, donc 0 alerte sur le vrai dataset). Après synchronisation avec BLACKLIST_IPS = IPS[:12] du générateur : {'192.168.1.1', ..., '192.168.1.12'}. Vérification : test_ip_set_analysis adapté, 12 détections réelles sur le dataset 12 Go.

1 choix technique argumenté de Dev A, Dev B et Dev C.

10. Répartition du travail

Le tableau 4 établit la répartition des contributions techniques et transversales.

Tableau 4 - Contributions individuelles

Équipier	Périmètre principal	Fichiers / livrables	Commits
Mniangadou (A)	Ingestion, parser, dataset, bench	src/ingestion/* ; src/utils/types.py ; scripts/generate_logs.py	[10] commits
Marlon (B)	Pipeline fonctionnel, CLI, qualité	src/pipeline/* ; main.py ; outils qualité	[3] commits
Baptiste (C)	Analytics, règles, audit, figures	src/analytics/* ; src/engine/*	[1] commits

Workflow Git-Flow : 3 branches feature (feature/dev-a ingestion, MP-pipeline Dev B, Branche_Baptiste Dev C), PR croisées sur main, revue de code avant merge. 40 tests pytest communs validés à chaque merge. Commits visibles via git log : 10 commits Dev A, 3 commits Dev B, 1 commit Dev C + revues sur l'ensemble.

Préciser les relectures de PR, l'intégration, la revue croisée et les contributions aux tests.

11. Conclusion

11.1 Bilan des objectifs

L'architecture combine lecture incrémentale, transformation fonctionnelle, agrégation et moteur de règles. Le tableau d'autoévaluation en annexe F associe chaque exigence du barème à une preuve technique et à la section correspondante.

11.2 Perspectives

Des extensions possibles concernent l'ingestion depuis des sockets ou Kafka, la configuration externe de nouvelles règles et une visualisation des alertes en temps réel. Elles prolongent le pipeline existant sans changer le principe du traitement itératif.

Pipeline validé sur 12,11 Go en un seul passage avec 42 Ko de RAM pic (ratio 301 106×). 40 tests pytest passants en 0,19 s. 2 règles de sécurité actives (bruteforce + blacklist). Tous les critères du barème couverts, preuves jointes en annexes A à F.

Annexes

A. Dépôt Git

Dépôt indiqué dans le cahier des charges du rapport :

https://github.com/Mamadouedu/projetspython

B. Dépendances Python

Python 3.12.3. Contenu de requirements.txt : pytest>=8.0 (tests), black>=24.0 et ruff>=0.4 (qualité), matplotlib>=3.8 (figures du rapport via scripts/make_figures.py). Le PIPELINE principal (main.py + src/) n'a AUCUNE dépendance runtime externe à la bibliothèque standard — le pipeline utilise uniquement collections, itertools, json, re, pathlib, enum, typing, dataclasses, argparse, tracemalloc.

C. Exemples de logs

Présenter au moins une ligne valide, une ligne invalide et un exemple d'événement de connexion.

Extrait C.1 - Lignes représentatives

```
2026-11-01T00:00:00Z | INFO | 999.999.999.999 | user=admin | {"action": "login"}
2026-11-01T00:00:00.520403Z | INFO | 192.168.3.7 | user=user_037 | {"action": "login", "status": "ok", "bytes": 6240}
2026-11-01T00:00:00.993548Z | INFO | 192.168.6.7 | user=user_045 | raw event 9928
2026-11-01T00:00:02.032704Z | INFO | 192.168.1.4 | user=user_048 | raw event 7924
2026-11-01T00:00:02.424902Z | ERROR | 192.168.10.2 | user=user_006 | {"action": "login", "status": "failed", "bytes": 23626}
2026-11-01T00:00:05.313725Z | WARNING | 192.168.13.8 | user=user_034 | {"action": "edit", "status": "ok", "bytes": 5264}
2026-11-01T00:00:04.781019Z | INFO | 192.168.10.15 | user=user_036 | {"action": "login", "status": "ok", "bytes": 3103}
2026-11-01T00:00:06.265959Z | CRITICAL | 192.168.7.6 | user=user_055 | {"action": "view", "status": "failed", "bytes": 29814}
2026-11-01T00:00:06.976928Z | INFO | 192.168.6.13 | user=user_042 | {"action": "upload", "status": "ok", "bytes": 42569}
2026-11-01T00:00:03.278360Z | INFO | 192.168.7.12 | user=user_010 | {"action": "view", "status": "ok", "bytes": 42043}
2026-11-01T00:00:09.193458Z | INFO | 192.168.2.10 | user=user_014 | raw event 6168
2026-11-01T00:00:07.271531Z | INFO | 192.168.1.42 | user=admin | {bad json, not valid}
2026-11-01T00:00:13.064770Z | INFO | 192.168.11.13 | user=user_056 | (192.168.13.7, 10387, HTTP)
2026-11-01T00:00:05.533675Z | INFO | 192.168.12.11 | user=user_057 | {"action": "view", "status": "ok", "bytes": 14473}
2026-11-01T00:00:16.766305Z | INFO | 192.168.2.6 | user=user_055 | {"action": "upload", "status": "ok", "bytes": 10584}
2026-11-01T00:00:15.193071Z | INFO | 192.168.11.6 | user=user_038 | (192.168.7.15, 64664, TCP)
2026-11-01T00:00:14.596081Z | WARNING | 192.168.8.4 | user=user_049 | raw event 2827
2026-11-01T00:00:09.590552Z | INFO | 192.168.8.2 | user=user_032 | raw event 9317
2026-11-01T00:00:20.184571Z | INFO | 192.168.14.13 | user=user_038 | {"action": "view", "status": "ok", "bytes": 10687}
2026-11-01T00:00:14.923381Z | INFO | 192.168.1.1 | user=user_038 | {"action": "login", "status": "ok", "bytes": 7431}
2026-11-01T00:00:22.723775Z | WARNING | 192.168.9.6 | user=user_015 | {"action": "delete", "status": "ok", "bytes": 5261}
2026-11-01T00:00:07.918850Z | INFO | 192.168.4.8 | user=user_008 | raw event 3705
2026-11-01T00:00:11.848121Z | INFO | 192.168.6.14 | user=user_059 | {"action": "upload", "status": "ok", "bytes": 45311}
2026-11-01T00:00:11.063827Z | CRITICAL | 192.168.11.2 | user=user_028 | (192.168.13.4, 8954, UDP)
2026-11-01T00:00:12.053462Z | INFO | 192.168.7.4 | user=user_037 | {"action": "login", "status": "ok", "bytes": 46489}
2026-11-01T00:00:21.699817Z | ERROR | 192.168.9.18 | user=user_004 | {"action": "view", "status": "failed", "bytes": 43942}
2026-11-01T00:00:19.158699Z | WARNING | 192.168.13.15 | user=user_015 | raw event 7669
2026-11-01T00:00:12.726961Z | INFO | 192.168.12.8 | user=user_026 | {"action": "upload", "status": "ok", "bytes": 3650}
2026-11-01T00:00:25.368787Z | INFO | 192.168.11.17 | user=user_046 | {"action": "download", "status": "ok", "bytes": 7261}
2026-11-01T00:00:15.189930Z | INFO | 192.168.12.8 | user=user_011 | {"action": "logout", "status": "ok", "bytes": 5040}
2026-11-01T00:00:20.964530Z | INFO | 192.168.2.7 | user=user_041 | (192.168.1.8, 64528, TCP)
```

D. Sortie complète du benchmark mémoire

Mesures tracemalloc sur Windows 11, Python 3.12.3. Dataset sample_large.log = 12,11 Go (119 510 991 lignes). Mode read-only (yield + yield from) : RAM pic = 42 Ko, débit = 28,4 Mo/s, ratio fichier/RAM = 301 106×. Commande : python scripts/bench_memory.py --input data/sample_large.log --read-only. Voir BENCH.md pour le tableau complet (small 5,2 Mo => 38 Ko ; large 2,75 Go => 61 Ko).

E. Commandes CLI utiles

Extrait E.1 - Installation, exécution et tests

```
# 1) Setup de l'environnement
python -m venv venv
.\venv\Scripts\Activate.ps1 # Windows PowerShell
pip install -r requirements.txt

# 2) Génération du dataset (12 Go, ~16 min, reproductible)
python scripts/generate_logs.py --out data/sample_large.log --gb 12 --seed 42

# 3) Exécution du pipeline complet (lecture + analyse + alertes)
python main.py --input data/sample_large.log --rules bruteforce,blacklist --audit

# 4) Tests unitaires (40 tests)
python -m pytest tests/ -v

# 5) Benchmark mémoire (preuve de non-saturation RAM)
python scripts/bench_memory.py --input data/sample_large.log --read-only

# 6) Qualité du code (PEP8 + formatage)
python -m ruff check src/ tests/
python -m black --check src/ tests/
```